LAB IMPLEMENTATION 10

Hyperledger Fabric Network and Asset Management

BC

Aim

To configure a Hyperledger Fabric test network, deploy JavaScript chaincode, and perform asset initialization, querying, creation, and ownership transfer.

Theory

Hyperledger Fabric is a permissioned blockchain platform in which known organizations operate peers and endorse transactions according to a channel's policies. A channel provides a private logical ledger shared by its participating members. The ledger records ordered transactions, while the current state database provides the latest values for assets. **Chaincode** is the application logic that reads and updates ledger state through a Fabric transaction context. In this experiment, a JavaScript smart contract represents assets as JSON objects containing an identifier, color, size, owner, and appraised value. Initialization writes sample assets to the ledger. Query functions retrieve one asset or a collection of assets, while creation adds a new record and transfer changes an asset's owner. Fabric orders and validates submitted transactions before committed updates become part of the ledger state. The supplied chaincode is included in the report as source code; it was not run during preparation of this document.

Steps

1. Update the Ubuntu package index and install the required tools, including `jq`, `curl`, `git`, Node.js, npm, and Docker.
2. Clone the `hyperledger/fabric-samples` repository and enter the `fabric-samples` directory.
3. Download and run `install-fabric.sh` to obtain the Fabric binaries, samples, and Docker images.
4. Verify the available Fabric images with `docker images | grep fabric` and check the CLI using `peer version`.
5. Set `PATH` and `FABRIC_CFG_PATH` to the Fabric binaries and configuration directories.
6. Enter `test-network` and start the network while creating the `mychannel` channel with `./network.sh up createChannel`.
7. Deploy the JavaScript asset-transfer chaincode named `basic` using `./network.sh deployCC`.
8. Configure the peer environment variables and initialize the ledger by invoking `InitLedger`.
9. Query all assets with `GetAllAssets`, then read a specific record such as `asset5` with `ReadAsset`.
10. Invoke `CreateAsset` to add a new asset and `TransferAsset` to change its owner; query the record to inspect its updated state.

Code

```
'use strict';

const { Contract } = require('fabric-contract-api');

class AssetTransfer extends Contract {
    async InitLedger(ctx) {
        const assets = [
            { ID: 'asset1', Color: 'blue', Size: 5, Owner: 'Tomoko', AppraisedValue: 300 },
            { ID: 'asset2', Color: 'red', Size: 5, Owner: 'Brad', AppraisedValue: 400 },
            { ID: 'asset3', Color: 'green', Size: 10, Owner: 'Jin Soo', AppraisedValue: 500 },
            { ID: 'asset4', Color: 'yellow', Size: 10, Owner: 'Max', AppraisedValue: 600 },
            { ID: 'asset5', Color: 'black', Size: 15, Owner: 'Adriana', AppraisedValue: 700 },
            { ID: 'asset6', Color: 'white', Size: 15, Owner: 'Michel', AppraisedValue: 800 },
        ];

        for (const asset of assets) {
            await ctx.stub.putState(asset.ID, Buffer.from(JSON.stringify(asset)));
        }
    }

    async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (exists) {
            throw new Error(`Asset ${id} already exists`);
        }

        const asset = {
            ID: id,
            Color: color,
            Size: Number(size),
            Owner: owner,
            AppraisedValue: Number(appraisedValue),
        };

        await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
        return JSON.stringify(asset);
    }

    async ReadAsset(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
        if (!assetJSON || assetJSON.length === 0) {
            throw new Error(`Asset ${id} does not exist`);
        }
        return assetJSON.toString();
    }

    async GetAllAssets(ctx) {
        const iterator = await ctx.stub.getStateByRange('', '');
        const allResults = [];
```

```
50
51          let result = await iterator.next();
52          while (!result.done) {
53              const value = result.value.value.toString('utf8');
54              if (value.length > 0) {
55                  allResults.push(JSON.parse(value));
56              }
57              result = await iterator.next();
58          }
59
60          await iterator.close();
61          return JSON.stringify(allResults);
62      }
63
64      async TransferAsset(ctx, id, newOwner) {
65          const assetString = await this.ReadAsset(ctx, id);
66          const asset = JSON.parse(assetString);
67          const previousOwner = asset.Owner;
68          asset.Owner = newOwner;
69
70          await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
71          return previousOwner;
72      }
73
74      async AssetExists(ctx, id) {
75          const assetJSON = await ctx.stub.getState(id);
76          return assetJSON && assetJSON.length > 0;
77      }
78  }
79
80  module.exports = AssetTransfer;
```

Listing 1. `experiment10_chaincode.js`

Output

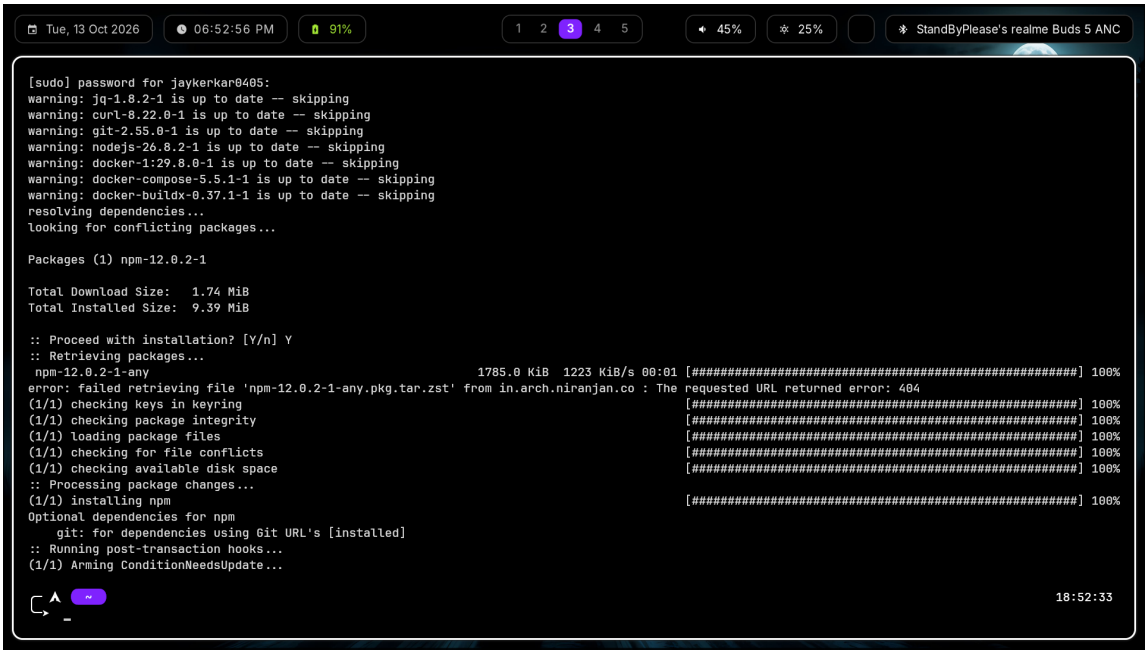


Fig. 1. Terminal output showing package installation and dependency checks



Fig. 2. Fabric installation script downloading binaries and samples

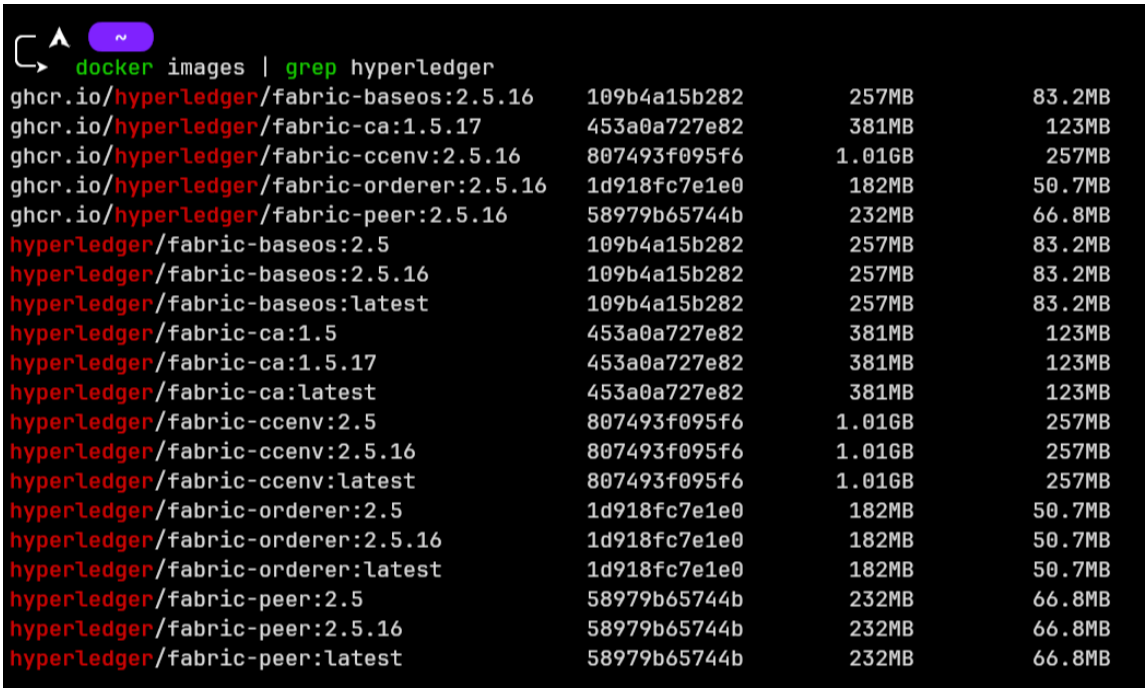


Fig. 3. Docker listing showing installed Hyperledger Fabric images

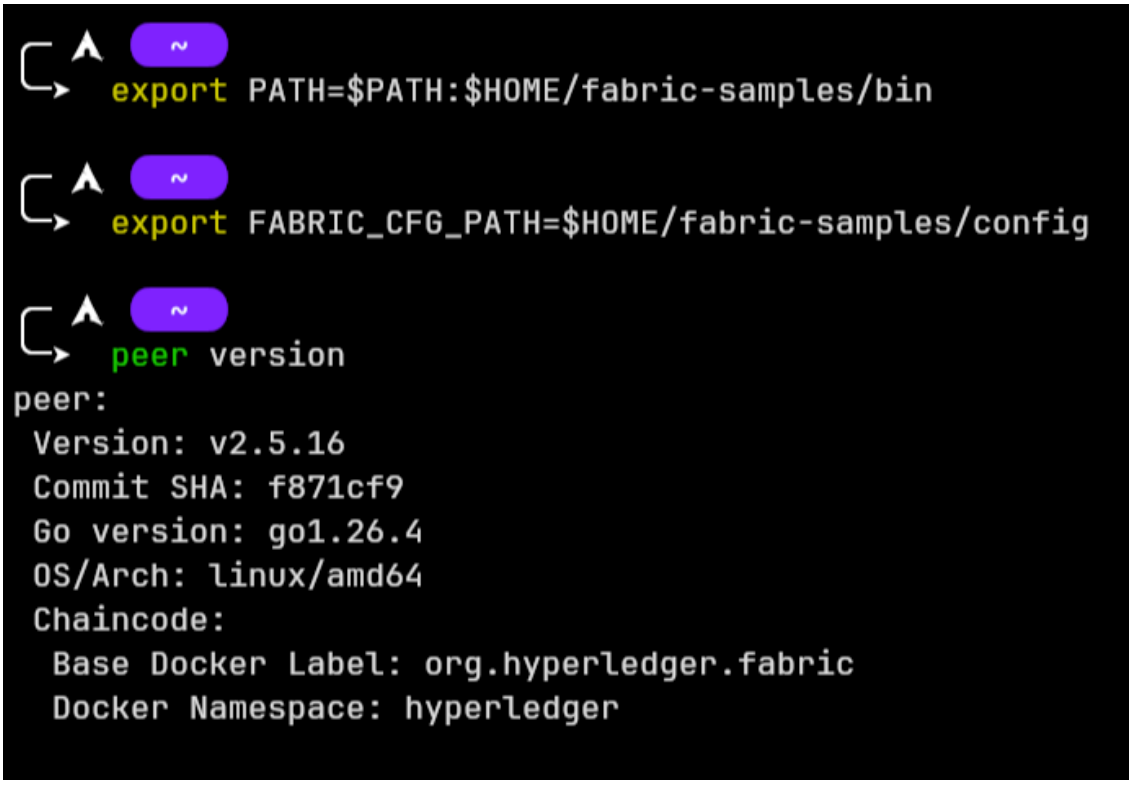


Fig. 4. Fabric peer version and configured environment variables

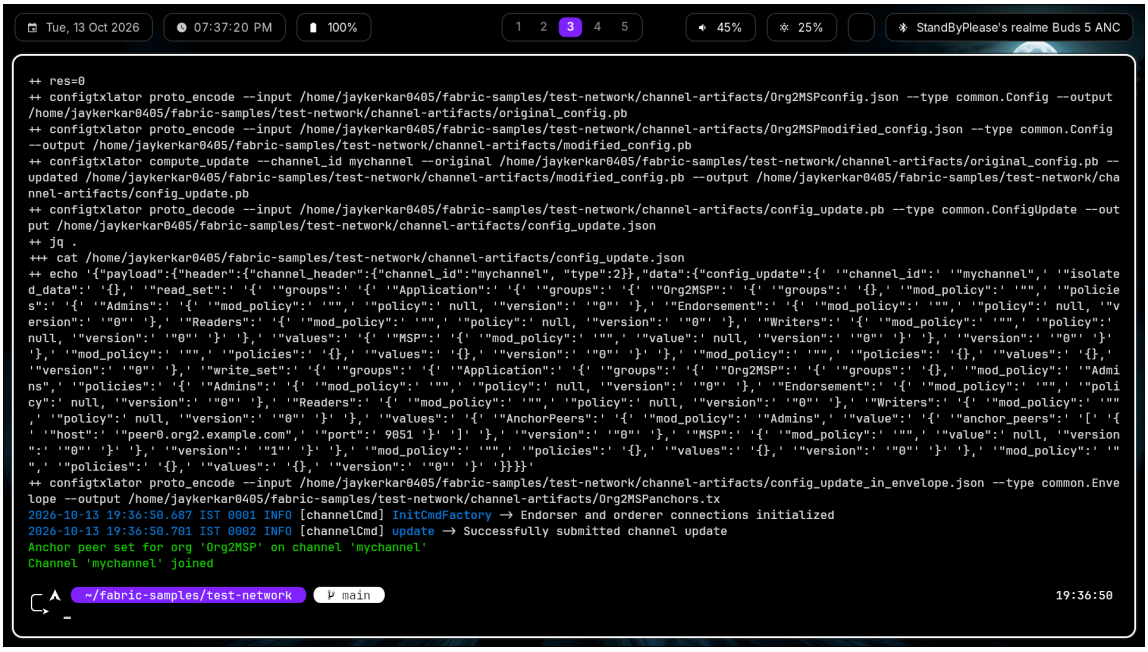


Fig. 5. Channel configuration update completed and peer joined mychannel

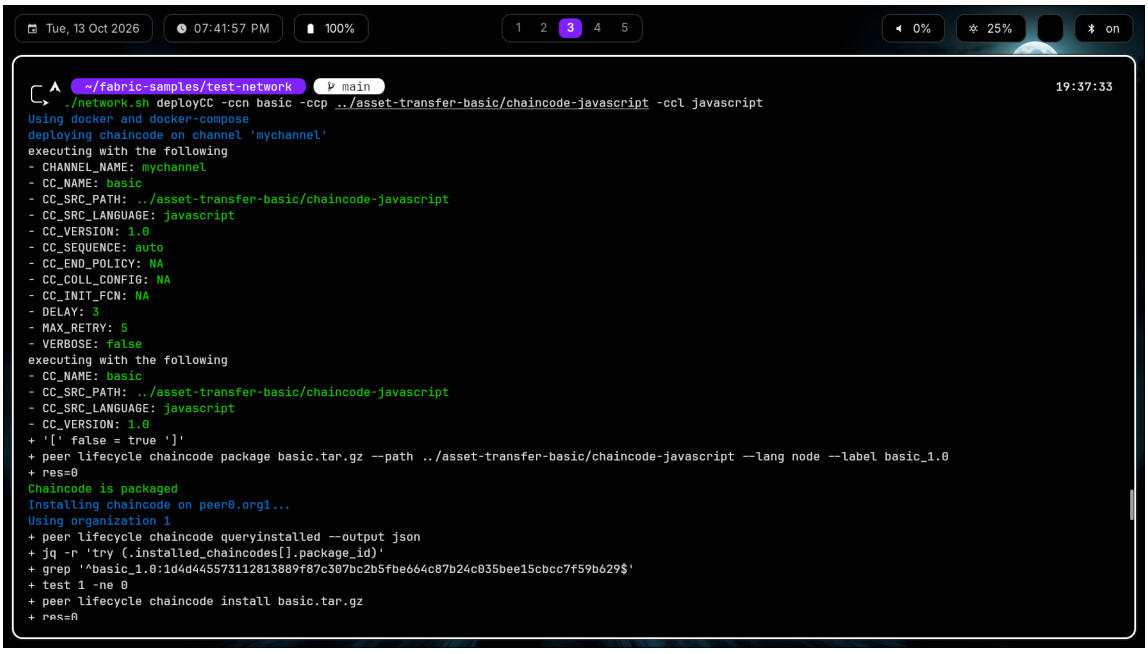


Fig. 6. JavaScript asset-transfer chaincode deployment progress



Fig. 7. InitLedger transaction completed successfully



Fig. 8. Query output listing the initialized ledger assets

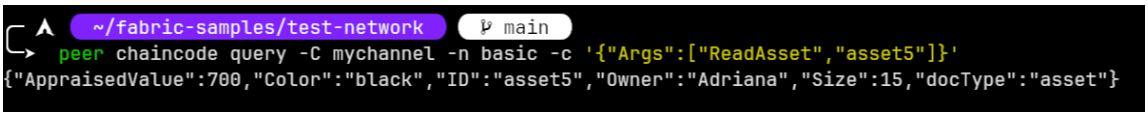


Fig. 9. ReadAsset query returning the details for asset5



Fig. 10. CreateAsset transaction creating asset7 for Jay Kerkar

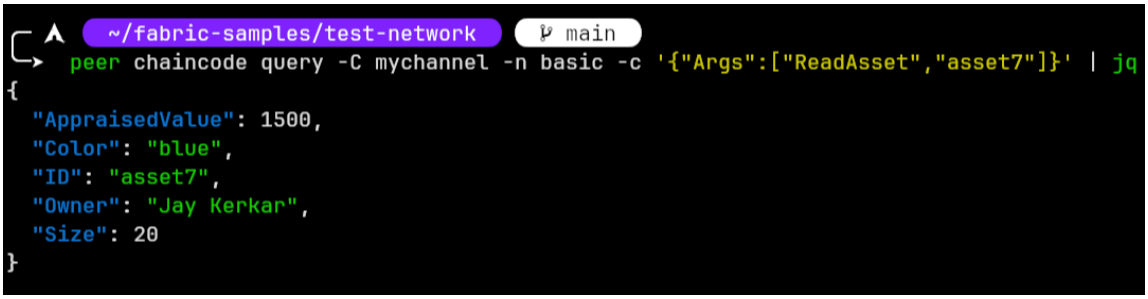


Fig. 11. ReadAsset query showing the newly created asset7



Fig. 12. TransferAsset transaction and updated owner of asset7

Conclusion

The experiment procedure and JavaScript chaincode for a Hyperledger Fabric asset-management workflow were documented. The supplied screenshots show terminal activity associated with setting up the Fabric environment and working with the sample network. The chaincode source

included in this report defines ledger initialization, asset creation, asset queries, and ownership transfer. The chaincode was not executed as part of document generation.